Managed Identities for Azure Logic Apps & Functions — Authenticating to Microsoft 365 Without Stored Credentials

Implementation Solution Guide

Version 1.0.0 · 2026-07-17

Contents

Executive Summary — Managed Identities for Azure Logic Apps & Functions (Authenticating to Microsoft 365 Without Stored Credentials) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Managed Identities for Azure Logic Apps & Functions (Authenticating to Microsoft 365 Without Stored Credentials) 5

What a managed identity is . . . 5

Why this matters . . . 5

System-assigned vs user-assigned managed identity . . . 5

How it works (the pattern) . . . 6

Where mailbox scoping fits (point to product #4 — do not reproduce it) . . . 6

Concrete mailbox applications (point to #1 / #2 / #3 — do not reproduce them) . . . 6

Benefits . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Managed Identities for Azure Logic Apps & Functions 8

2.1 An Azure subscription with Logic Apps and/or Functions . . . 8

2.2 Ability to enable a managed identity on the compute . . . 8

2.3 Entra rights to grant application permissions and admin consent . . . 8

2.4 Microsoft.Graph PowerShell modules . . . 8

2.5 The target Microsoft 365 workload(s) and the exact application permission . . . 9

2.6 Logic Apps hosting model — Consumption vs Standard *(verify)* . . . 9

2.7 National / sovereign cloud endpoints *(verify)* . . . 9

2.8 Change control, pilot workflow, and rollback . . . 9

Checklist . . . 10

Global Requirements — Azure Solutions 10

G1. A Microsoft 365 / Entra tenant . . . 10

G2. Administrative roles (least privilege) . . . 10

G3. Licensing baseline . . . 11

G4. Network connectivity to Microsoft 365 / Azure endpoints . . . 11

G5. Change control & maintenance window . . . 11

G6. Backup / rollback posture . . . 11

3. Implementation Guide — Managed Identities for Azure Logic Apps & Functions 12

Naming (set once, reuse verbatim) . . . 12
Step 1 — Enable a managed identity on the compute . . . 12
Step 2 — Grant least-privilege application Graph permissions with admin consent . . . 13
Step 3 — Wire a Logic App to call Graph with the managed identity (no connector, no secret) . . . 14
Step 4 — Wire an Azure Function to call Graph with the managed identity (code) . . . 14
Step 5 — Validate least-privilege and the token flow . . . 15
Step 6 — Migrate secret-based flows onto the identity and retire the credentials . . . 15
Step 7 — Expand across the estate . . . 15
5. Testing Guide — Managed Identities for Azure Logic Apps & Functions 16
Test 1 — Managed identity enabled on the compute . . . 16
Test 2 — Least-privilege app-role granted (and nothing broader) . . . 16
Test 3 — Logic App authenticates to Graph with the identity . . . 16
Test 4 — Azure Function authenticates to Graph with the identity . . . 16
Test 5 — No stored credential remains on migrated flows . . . 17
Test 6 — Secret-based flow migrated with parity . . . 17
Test 7 — Least-privilege boundary holds . . . 17
Test 8 — Estate inventory report . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Managed Identities for Azure Logic Apps & Functions 18
A. Managed identity not enabled / not found . . . 18
B. Permission / admin consent missing . . . 18
C. 403 / insufficient privileges on a mailbox call . . . 19
D. Token audience / scope errors . . . 19
E. User- vs system-assigned confusion . . . 19
F. Azure Functions credential not resolving . . . 19
G. Connector action won't use a managed identity . . . 20
H. Stored credential still flagged after migration . . . 20
I. Diagnostics quick reference . . . 20
J. When to escalate . . . 20
7. Best Practices — Managed Identities for Azure Logic Apps & Functions 21
8. Deliverables Summary — Managed Identities for Azure Logic Apps & Functions 21
Documentation . . . 21
PowerShell scripts (Microsoft.Graph) . . . 22
What the buyer receives . . . 22
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Managed Identities for Azure Logic Apps & Functions (Authenticating to Microsoft 365 Without Stored Credentials)

White-label · Azure Solutions

Automations that reach into Microsoft 365 too often run on a **stored service-account connection** or a **client secret** sitting in the workflow — credentials that must be rotated, can leak, and grant standing access far beyond the one task they exist for. A **managed identity** removes the credential entirely: Azure issues and rotates it, and you grant it exactly the application permissions it needs. This solution is the **pattern reference** for authenticating Azure **Logic Apps** and Azure **Functions** to Microsoft 365 (Microsoft Graph and related resources) with managed identities — and for migrating existing secret-based automations off stored credentials.

This is a complete, **deploy-it-today** package written for an administrator with no prior managed-identity experience.

What you get

- A plain-language **Solution Overview** of managed identities — system-assigned vs user-assigned and when to choose each, and the Logic Apps and Functions authentication patterns.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite.
- An **Implementation Guide** of precise, ordered steps — enable the identity, grant a **least-privilege** application permission with admin consent, call Graph from a **Logic App** (no connector, no secret), call Graph from a **Function** (with a working code example), validate, and migrate.
- Five **PowerShell scripts** (Microsoft.Graph) for identity/app-role discovery, least-privilege validation, an estate inventory + CSV, a stored-credential finder that produces the migration worklist, and a `-WhatIf`-gated app-role assignment helper.
- **Testing** and **Troubleshooting** guides for confident rollout.

What this delivers

Azure Logic Apps and Functions authenticating to Microsoft Graph through **managed identities** and **least-privilege application permissions** — no stored service-account passwords, no client secrets — with a repeatable path to migrate existing secret-based flows and retire the credentials. Mailbox scoping is handled by companion product #4; the mailbox-specific workloads by products #1/#2/#3. This document is the general pattern that underpins them all, across both compute types.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — pilot one workflow, prove least privilege and parity, then expand across the estate, driving the stored-credential worklist to zero.

Outcome

A credential-free authentication standard for your Azure automations — proven on a pilot workflow across both Logic Apps and Functions before you scale, deployable by a single administrator.

1. Solution Overview — Managed Identities for Azure Logic Apps & Functions (Authenticating to Microsoft 365 Without Stored Credentials)

What a managed identity is

A **managed identity** is an identity that Azure creates and manages for a compute resource — an Azure Logic App or an Azure Function — so the resource can authenticate to other Azure and Microsoft 365 services **without any credential stored in the workflow or the code**. Azure issues the identity, rotates its underlying secret automatically, and removes it when the resource is deleted. Your automation asks Azure for a token at runtime; nothing is ever written into a connection string, an app setting, or a workflow definition. This solution is the **pattern reference** for using managed identities to authenticate **both** Azure Logic Apps **and** Azure Functions to **Microsoft 365** (Microsoft Graph and related resources), and for **migrating existing automations off stored service-account connections and client secrets**.

Why this matters

Automations that reach into Microsoft 365 too often run on a **stored service-account connection** (a licensed user whose password sits in a saved connector connection) or a **client secret** pasted into the workflow or app settings. Those credentials must be rotated on a schedule, can leak into logs, source control, or exports, and grant **standing access** that survives long after the automation stops being used — a service account with a saved password can be reused by anyone who obtains it, and it typically carries far more access than the one task needs. A managed identity removes the credential from the picture entirely: there is nothing to rotate, nothing to leak, and access is bounded by the **application permissions** you explicitly grant. Combined with **least-privilege application permissions**, the automation can do exactly one job and no more.

System-assigned vs user-assigned managed identity

There are two kinds, and choosing correctly is the first design decision:

- **System-assigned managed identity** — created **on** a single Logic App or Function, tied to that resource's lifecycle (deleted with it), and usable only by that resource. Choose it when a workflow's identity should live and die with the workflow and is not shared. Simple, but every resource gets its **own** identity, so app-permission grants and clean-up multiply across the estate.
- **User-assigned managed identity** — a **standalone** Azure resource you create once and then **attach to many** Logic Apps and Functions. Choose it when several workflows should share one identity and one set of granted permissions. Grant the application permissions **once** to the shared identity, and every workflow attached to it inherits them — one object to audit, one to review, one to remove. For a **multi-workflow estate**, a shared user-assigned identity is usually the simpler operational model; a single system-assigned identity is fine for a one-off, self-contained workflow.

> ***Verify against current Microsoft documentation:*** *exact support for user-assigned identities in* ***Logic Apps Consumption vs Standard****, and in the various Azure Functions hosting plans, differs — confirm the compute type in scope supports the identity kind you choose before you commit to the design.*

How it works (the pattern)

1. **Enable a managed identity** on the Logic App or Function — a **system-assigned** identity toggled on for one resource, or a **user-assigned** identity created once and attached to many.

2. **Grant the identity least-privilege application permissions** — assign the specific **application** Microsoft Graph permission(s) the task needs (an **app-role assignment** to the managed identity's service principal) and grant **admin consent**. Least privilege by default: grant only the narrow permission the workflow uses, never a broad tenant-wide grant "to be safe."

3. **The compute calls Microsoft Graph with the identity** — a Logic App uses the built-in **managed-identity authentication** on its HTTP action (no connector, no secret); a Function uses `DefaultAzureCredential` / `ManagedIdentityCredential` in code to acquire a token. Azure returns a token scoped to the granted permissions; the call runs as the application.

4. **Validate and migrate** — confirm the identity authenticates and is least-privilege, then walk existing secret/service-account flows onto the same pattern and retire the stored credentials.

Where mailbox scoping fits (point to product #4 — do not reproduce it)

Application Graph permissions like `Mail.Read` are, by default, **tenant-wide** — the app can read *every* mailbox. That is almost never what you want. **Restricting an application permission to a single mailbox (or a defined set)** is a distinct discipline covered in full by the companion product **"Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online" (product #4)**. This solution grants permissions **least-privilege** and hands mailbox scoping off to #4 rather than reproducing it — whenever a step here grants a mailbox permission, apply #4's scoping before considering the access production-ready.

Concrete mailbox applications (point to #1 / #2 / #3 — do not reproduce them)

This is the broad pattern reference; the mailbox-specific siblings show it applied end-to-end and are the place to go for those workloads:

- **Product #1 — "Exchange Online Mailbox Automation with Logic Apps Managed Identity — Replacing the O365 Connector with Microsoft Graph"**: replacing a stored Office 365 Outlook connector with managed-identity Graph calls.
- **Product #2 — "Event-Driven Mail Processing with Microsoft Graph Change Notifications (Logic Apps + Managed Identity)"**: reacting to new mail via Graph change-notification subscriptions.
- **Product #3 — "Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)"**: scheduled polling of a mailbox without a service account.

Use those for the mailbox specifics; use this document for the **general managed-identity-to-M365 authentication pattern** that underpins all of them, across **both** Logic Apps and Functions.

Benefits

• **No credential to store, rotate, or leak** — Azure issues and rotates the identity; nothing lives in the workflow, app settings, or code.

• **Least-privilege by design** — the automation holds only the application permissions you explicitly grant, not a service account's full access.

• **One pattern, two compute types** — the same managed-identity approach authenticates Logic Apps and Functions, so the estate is consistent.

• **A clean migration target** — every stored-secret and service-account flow has a defined path onto the identity model.

• **Simpler audit** — a user-assigned identity gives one object to review and one set of grants to justify across many workflows.

Limitations (be honest with buyers)

• A managed identity works only **inside Azure** — it authenticates Azure compute (Logic Apps, Functions) to services that accept Entra tokens. It does not replace credentials for software running outside Azure.

• **Application permissions are tenant-wide until scoped** — `Mail.Read` granted to the identity can read every mailbox until you constrain it (see product #4). Granting the permission is not the same as scoping it.

• **Not every legacy connector action supports a managed identity** — some Office 365 / Outlook connector triggers and actions may not accept managed-identity auth, which is one reason the pattern favours **direct Graph calls**. *Verify against current Microsoft documentation which specific connector operations support managed identity.*

• **Consumption vs Standard Logic Apps and the various Functions hosting plans differ** in managed-identity and networking behaviour. *Verify the exact capabilities for the compute type and plan in scope against current Microsoft documentation.*

• **National / sovereign clouds use different endpoints** (Graph, login/authority, and resource URLs). *Verify the correct endpoints for the cloud in scope against current Microsoft documentation.*

• Granting **admin consent** for application permissions requires a **Privileged Role Administrator / Global Administrator**; this is a governed action, not a self-service one.

The standard deployment model

Enable a managed identity (system- or user-assigned) → grant least-privilege application permissions with admin consent → wire a Logic App to call Graph with the identity → wire a Function to call Graph with the identity → validate least-privilege and token flow → migrate secret-based flows onto the identity and retire the credentials. *Repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, piloting on one workflow before rolling the pattern across the estate, with the stored-credential inventory as the migration worklist.*

2. Requirements Guide — Managed Identities for Azure Logic Apps & Functions

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-azure-solutions.md`. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim.

2.1 An Azure subscription with Logic Apps and/or Functions

Description: At least one Azure subscription under the Microsoft Entra tenant, with the Azure Logic Apps and/or Azure Functions resource(s) that will call Microsoft 365 (either existing or to be created). **Why:** A managed identity is enabled **on** a Logic App or Function; there must be a compute resource to host it. **Verify:** `portal.azure.com` → **Logic apps** and/or **Function App**; run `MIM365-Discovery.ps1`. **Implement-if-missing:** Create the Logic App (Consumption or Standard) or Function App in the subscription in scope, and name it per your convention (e.g. `Graph Automation – Contoso – Pilot`).

2.2 Ability to enable a managed identity on the compute

Description: Rights to turn on a **system-assigned** identity on the Logic App/Function, and/or to **create and attach** a **user-assigned** managed identity. **Why:** The whole pattern rests on the compute having an identity; enabling it is a write operation on the resource. **Verify:** On the resource → **Identity** blade you can toggle **System assigned = On** or add a **User assigned** identity; `MIM365-Discovery.ps1` reports whether each compute already has an identity. **Implement-if-missing:** Grant **Contributor** (or a role with `Microsoft.Web/sites/write` / the Logic App equivalent) on the resource; for a user-assigned identity, **Managed Identity Contributor** to create it and **Managed Identity Operator** to attach it.

2.3 Entra rights to grant application permissions and admin consent

Description: An account able to assign **application** Microsoft Graph permissions (app-role assignments) to the managed identity's service principal **and** grant **admin consent**. **Why:** Application permissions do not take effect until an app-role is assigned to the identity and admin consent is granted; consent is a privileged, tenant-governed action. **Verify:** Entra → **Roles & admins** confirms the operating account holds **Privileged Role Administrator** or **Global Administrator** (for consent) and **Application Administrator**/**Cloud Application Administrator** (to manage app-role assignments); `MIM365-Discovery.ps1` reports current app-role assignments per identity. **Implement-if-missing:** Assign the minimum roles at the tenant scope; prefer **PIM eligible** activation over standing access.

2.4 Microsoft.Graph PowerShell modules

Description: The **Microsoft.Graph** PowerShell SDK on the operator workstation, used by every script in this solution. **Why:** Discovery, validation, reporting, the stored-credential inventory, and the app-role helper all call Microsoft Graph via the SDK. **Verify:** `Get-Module Microsoft.Graph -ListAvailable`; `MIM365-Discovery.ps1` fails fast if the module is missing. **Implement-if-missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`. Connect with the least scopes each script names in its

header (e.g. `Application.Read.All`, `AppRoleAssignment.ReadWrite.All`, `Directory.Read.All`).

2.5 The target Microsoft 365 workload(s) and the exact application permission

Description: A decision on **which** Microsoft 365 workload the automation calls (Exchange Online / Outlook mail, users, groups, files, etc.) and the **specific least-privilege application Graph permission** it requires. **Why:** The app-role you assign must match the task; granting more than the one permission the workflow uses defeats the purpose. **Verify:** The workflow's Graph call is known (endpoint + method), and the single application permission that authorizes it is identified. **Implement-if-missing:** Map the Graph call to its required application permission and record it as the only grant. *Verify the exact permission name/scope for your call against current Microsoft documentation — do not assume a permission name.* For **mailbox** permissions, plan to **scope** the grant to specific mailboxes per product #4 rather than leaving it tenant-wide.

2.6 Logic Apps hosting model — Consumption vs Standard *(verify)*

Description: Awareness of whether the Logic App is **Consumption** or **Standard**, because managed-identity support, the workflow authoring surface, and networking differ. **Why:** A step that works in one model may differ in the other; the design must match the hosting model in scope. **Verify:** `portal.azure.com` → the Logic App **Overview** shows the plan/type. *Confirm the managed-identity behaviour for that model against current Microsoft documentation.* **Implement-if-missing:** Choose the hosting model deliberately; if a required identity feature is unsupported on the current model, plan the workflow on the model that supports it. **Flag as verify — do not assert Consumption/Standard specifics.**

2.7 National / sovereign cloud endpoints *(verify)*

Description: If the tenant is in a national/sovereign cloud (e.g. US Government), the Graph, login/authority, and resource endpoints differ from commercial. **Why:** The managed-identity token must be requested for the correct **resource/audience** and the code/action must target the correct **Graph endpoint**, or calls fail with audience/authority errors. **Verify:** Confirm the cloud in scope and its endpoint set. *Verify the exact Graph/authority/resource URLs for that cloud against current Microsoft documentation.* **Implement-if-missing:** Configure the Logic App action / Function code and `DefaultAzureCredential` authority for the correct cloud. **Flag as verify — do not assert sovereign-cloud endpoints.**

2.8 Change control, pilot workflow, and rollback

Description: An approved change window, a named **pilot** Logic App/Function, and a documented rollback before migrating any production flow off stored credentials. **Why:** Switching a live automation's authentication can break it; piloting one workflow and keeping a rollback is the safety net. **Verify:** An approved change record, the pilot resource name, and a written back-out exist (per `global-requirements-azure-solutions.md` G5/G6). **Implement-if-missing:** Capture the current state (`MIM365-Report.ps1`), pilot on one workflow, and write the rollback (re-enable the previous connection) before cutting over production.

Checklist

- Azure subscription + Logic App and/or Function App — `MIM365-Discovery.ps1`
- Rights to enable system-assigned / create + attach user-assigned identity
- Entra rights to assign application Graph permissions **and** grant admin consent
- Microsoft.Graph PowerShell installed and connectable
- Target M365 workload + exact least-privilege application permission identified *(verify the permission name)*
- Logic Apps Consumption vs Standard understood *(verify)*
- National/sovereign cloud endpoints confirmed if applicable *(verify)*
- Change window, pilot workflow, and rollback documented

Global Requirements — Azure Solutions

These requirements are common to **every** Azure Solutions solution in the standard program model. They are the baseline platform prerequisites that any Azure/Microsoft 365 engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

> *Naming convention: every created object is named `<Purpose> – <Scope> – <Stage>` and reused verbatim throughout the engagement.*

G1. A Microsoft 365 / Entra tenant

- **Description:** An active Microsoft 365 / Microsoft Entra tenant for the cloud(s) in scope (commercial and/or government).
- **Why:** All identity, licensing, and service configuration is anchored to a tenant; nothing can be configured without one.
- **Verify:** Sign in to the relevant admin center (`https://admin.microsoft.com`) and confirm the tenant resolves and is active under **Settings** → **Org settings**.
- **Implement-if-missing:** Provision the tenant through the appropriate channel (commercial: standard; government: via an authorized AOS-G reseller).

G2. Administrative roles (least privilege)

- **Description:** Appropriately scoped admin roles for the work — Global Administrator for tenant-wide changes, and service-specific roles (Exchange, SharePoint, Teams, Azure Owner/Contributor) for day-to-day.
- **Why:** Every configuration and migration action requires a role with sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** In Entra → **Roles & admins**, confirm the operating accounts hold the required roles. In Azure → **Subscriptions** → **Access control (IAM)**, confirm RBAC assignments.

• **Implement-if-missing:** Assign the minimum roles needed; prefer eligible (PIM) activation over standing access where available.

G3. Licensing baseline

• **Description:** Sufficient Microsoft 365 and/or Azure licensing/credit for all in-scope users and workloads.

• **Why:** Features and capacity (mailboxes, SharePoint storage, AVD, VMs) are gated by license SKUs and subscription quota.

• **Verify: Billing** → **Licenses** for M365; **Subscriptions** → **Usage + quotas** for Azure. Compare assigned/available against the in-scope object count.

• **Implement-if-missing:** Procure the correct SKUs and request quota increases sized to the workload (add headroom for shared/resource objects).

G4. Network connectivity to Microsoft 365 / Azure endpoints

• **Description:** Reliable network egress to the required Microsoft 365 and Azure service endpoints (and the sovereign `.us` endpoints for government clouds), with sufficient bandwidth.

• **Why:** All cloud administration and any data movement depends on connectivity to the correct endpoint set; government clouds use distinct endpoints from commercial.

• **Verify:** Confirm reachability to the relevant endpoints (commercial `*.microsoft.com` / government `*.microsoft.us`, `*.azure.us`, `*.office365.us`). Measure sustained bandwidth against the data volume.

• **Implement-if-missing:** Open firewall/proxy allowances for the correct endpoint set; scale bandwidth or add ExpressRoute (matching the cloud — Gov ExpressRoute for Gov regions).

G5. Change control & maintenance window

• **Description:** An approved change-management process with scheduled maintenance windows and a stakeholder communications plan.

• **Why:** Cloud changes can be outward-facing and disruptive; controlled windows and comms reduce business impact and satisfy governance/compliance.

• **Verify:** Confirm an approved change record and a scheduled window for each disruptive action.

• **Implement-if-missing:** Stand up a lightweight change/approval process; schedule windows; prepare user comms templates.

G6. Backup / rollback posture

• **Description:** A defined backup and rollback position for any change — config export, data backup, and a documented back-out procedure before any disruptive action.

• **Why:** Cloud changes are not always reversible by default; a tested rollback is the safety net for cutovers and bulk operations.

• **Verify:** Confirm pre-change config/state is captured and a written rollback exists for each disruptive change.

• **Implement-if-missing:** Export current configuration/state; document and (where feasible) rehearse the rollback before executing the change.

3. Implementation Guide — Managed Identities for Azure Logic Apps & Functions

Follow these steps **in order**, in the **Azure portal** (`https://portal.azure.com`) and the **Entra admin center** (`https://entra.microsoft.com`). Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test. Contoso is the sample tenant throughout (`contoso.com`).

> ***Model:*** *Enable a managed identity → grant least-privilege application permissions with admin consent → wire a **Logic App** to call Graph with the identity → wire an **Azure Function** to call Graph with the identity → validate → migrate secret-based flows and retire the credentials. Pilot on one workflow first; keep a rollback for every cutover. Everything version/feature/licensing-specific — exact Graph permission names, whether a given connector action supports managed identity, Consumption-vs-Standard behaviour, national-cloud endpoints, and Azure SDK class/method names — is flagged _**verify against current Microsoft documentation**_. Do not treat those as asserted fact.*

Naming (set once, reuse verbatim)

• Shared user-assigned managed identity: `Graph Automation Identity – Contoso – Prod`

• Pilot Logic App: `Graph Automation LogicApp – Contoso – Pilot`

• Pilot Function App: `Graph Automation FuncApp – Contoso – Pilot`

• App-role assignment record (informal label for the grant): `Mail.Read App Grant – Graph Automation Identity – Prod`

> ***Choosing the identity kind (decide before Step 1):*** *use a **system-assigned** identity for a single, self-contained workflow whose identity should live and die with it; use the **shared user-assigned** identity* `Graph Automation Identity – Contoso – Prod` *when several Logic Apps/Functions should share one identity and one set of grants. This guide shows **both**.*

Step 1 — Enable a managed identity on the compute

Option A — System-assigned (per resource)

1. `portal.azure.com` → open the Logic App `Graph Automation LogicApp – Contoso – Pilot` (or the Function App).

2. Left menu → **Identity** → **System assigned** tab → set **Status = On** → **Save** → **Yes**.

3. Azure creates a service principal for the resource. Note the **Object (principal) ID** shown — you will grant the app-role to this ID.

Option B — User-assigned (shared across many workflows) — recommended for an estate

1. `portal.azure.com` → **Managed Identities** → **Create**. Subscription/Resource group as required; **Name:** `Graph Automation Identity – Contoso – Prod` → **Review + create** → **Create**.

2. Open the Logic App `Graph Automation LogicApp – Contoso – Pilot` → **Identity** → **User assigned** tab → **Add** → select `Graph Automation Identity – Contoso – Prod` → **Add**.

3. Repeat 2 for the Function App `Graph Automation FuncApp – Contoso – Pilot` and any other workflow that will share the identity.

4. On the managed identity's **Overview**, note its **Object (principal) ID** and **Client ID** — you will need the **Object ID** to grant the app-role and the **Client ID** in Function code that targets a specific user-assigned identity.

Run `MIM365-Discovery.ps1` to confirm which Logic Apps/Functions now have an identity and which app-roles (if any) are already assigned.

Step 2 — Grant least-privilege application Graph permissions with admin consent

Grant **only** the one application permission the task needs (least privilege). This example uses `Mail.Read`; substitute the exact permission your Graph call requires. *Verify the exact application permission name/scope for your call against current Microsoft documentation.*

1. Identify the managed identity's **service principal Object ID** from Step 1 (system-assigned: the resource's principal ID; user-assigned: the identity's Object ID).

2. Assign the **application** app-role to that principal and grant admin consent. The repeatable way is the helper script — **preview first**, then execute:

```
# Preview (default -WhatIf): shows the grant, changes nothing
.\MIM365-Set-AppRole.ps1 -ManagedIdentityObjectId '<identity object id>' -GraphAppRole 'Mail.Read'
# Apply for real (explicit):
.\MIM365-Set-AppRole.ps1 -ManagedIdentityObjectId '<identity object id>' -GraphAppRole 'Mail.Read' -Exe
```

The script resolves the Microsoft Graph service principal, finds the named **application** app-role, and creates the app-role assignment to the managed identity — the equivalent of granting the application permission and consenting to it for that identity.

1. Record the grant as `Mail.Read App Grant – Graph Automation Identity – Prod`. **Grant nothing else** — no extra "just in case" permissions.

2. **Scope mailbox permissions before production.** `Mail.Read` as granted is **tenant-wide** (every mailbox). Restrict it to the specific mailbox(es) the workflow needs using the companion product **#4 "Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online."** Do not consider a mailbox grant production-ready until it is scoped per #4. *(This guide does not reproduce #4's scoping steps.)*

Run `MIM365-Validate.ps1` to confirm the identity holds the intended least-privilege app-role and nothing broader.

Step 3 — Wire a Logic App to call Graph with the managed identity (no connector, no secret)

1. Open `Graph Automation LogicApp – Contoso – Pilot` → the workflow designer.

2. Add an **HTTP** action for the Graph call — e.g. **Method** `GET`, **URI** `https://graph.microsoft.com/v1.0/users/adele@contoso.com/messages?$top=5`. *Verify the Graph endpoint/host for your cloud against current Microsoft documentation (commercial vs national cloud differ).*

3. Expand **Add new parameter** → **Authentication** → **Authentication type = Managed identity**.

4. **Managed identity:** choose **System-assigned managed identity**, or the user-assigned `Graph Automation Identity – Contoso – Prod`.

5. **Audience:** `https://graph.microsoft.com` — the resource the token is requested for. *Verify the audience/resource URI for your cloud against current Microsoft documentation.*

6. **Save.** No connector, no client secret, and no stored connection is created — the action asks Azure for a token for the identity at run time.

7. **Run** the workflow (or trigger it) and confirm the HTTP action returns **200** with the expected Graph payload.

*If a step in an existing workflow uses the **Office 365 Outlook connector**, note that not every connector action supports managed-identity auth — the robust pattern is the **direct Graph HTTP call** shown here. *Verify which connector operations support managed identity against current Microsoft documentation.* The mailbox-specific siblings (#1 replace-the-connector, #2 change-notifications, #3 polling) show these Graph mail calls end-to-end.*

Step 4 — Wire an Azure Function to call Graph with the managed identity (code)

The Function acquires a token for the managed identity in code with `DefaultAzureCredential` (which resolves to the managed identity when running in Azure) or `ManagedIdentityCredential`, then calls Graph. Minimal, correct-shaped C# example — *verify the exact Azure SDK package names, class names, and method signatures against current Microsoft documentation, as they change between SDK versions:*

```
using Azure.Identity;              // Azure.Identity package — verify current version
using Microsoft.Graph;             // Microsoft.Graph SDK — verify current version/namespaces

// DefaultAzureCredential resolves to the Function's managed identity when running in Azure.
// For a SPECIFIC user-assigned identity, pass its Client ID so the right identity is used:
var credential = new DefaultAzureCredential(
    new DefaultAzureCredentialOptions
    {
        ManagedIdentityClientId = "<user-assigned identity Client ID>"  // omit for system-assigned
    });

// The Graph client uses the managed-identity token automatically. Scope/host differ by cloud —
// verify "https://graph.microsoft.com/.default" and the Graph base URL for your cloud.
var graph = new GraphServiceClient(credential,
    new[] { "https://graph.microsoft.com/.default" });

// Example least-privilege call (requires the Mail.Read app-role granted in Step 2):
var messages = await graph.Users["adele@contoso.com"].Messages
    .GetAsync(cfg => cfg.QueryParameters.Top = 5);
```

1. In the Function App `Graph Automation FuncApp – Contoso – Pilot`, ensure the managed identity from Step 1 is enabled/attached.

2. Add the Azure Identity and Microsoft Graph SDK packages to the project. *Verify the current package names/versions against current Microsoft documentation.*

3. Use the code shape above: build the credential, build the Graph client, make the least-privilege call. For a **user-assigned** identity, set `ManagedIdentityClientId` to its **Client ID**; for **system-assigned**, omit it.

4. Deploy and invoke the Function; confirm it returns the expected Graph result **without any secret in app settings or code**.

*The same pattern applies in other languages via that language's Azure Identity library (`ManagedIdentityCredential` / `DefaultAzureCredential`). *Verify the library and API names for your language against current Microsoft documentation.**

Step 5 — Validate least-privilege and the token flow

1. Run `MIM365-Validate.ps1` → confirms the target compute has a managed identity, holds the intended least-privilege app-role (e.g. `Mail.Read`) and **nothing broader**, and has **no leftover secret-based connection**.

2. Confirm the Logic App HTTP action (Step 3) and the Function (Step 4) both return **200 / expected data** with no stored credential.

3. Run `MIM365-Report.ps1` to capture the identity/permission inventory as the pre-migration baseline.

Step 6 — Migrate secret-based flows onto the identity and retire the credentials

1. Run `MIM365-Find-StoredCredentials.ps1` to produce the **migration worklist** — the Logic Apps/Functions still using stored service-account connections or client secrets.

2. For each item, on a pilot-first basis: enable the managed identity (Step 1), grant the same least-privilege permission (Step 2), re-point the workflow's Graph call to managed-identity auth (Step 3 / Step 4), and validate parity (Step 5) — the migrated flow must do exactly what the secret-based flow did.

3. Once parity is confirmed and the change window is approved, **remove the stored credential** — delete the saved connector connection and delete the client secret / service-account app setting. **Rollback:** if the migrated flow misbehaves, re-enable the previous connection until fixed.

4. Re-run `MIM365-Find-StoredCredentials.ps1` — the migrated workflow should no longer appear. Repeat until the worklist is empty.

Step 7 — Expand across the estate

1. Attach the shared `Graph Automation Identity – Contoso – Prod` to the remaining workflows that need the same permission (they inherit the existing grant — nothing new to consent).

2. For workflows needing a **different** permission, grant that one least-privilege app-role (Step 2) — scoping mailbox permissions per #4.

3. Re-validate each with `MIM365-Validate.ps1` and track the estate with `MIM365-Report.ps1` each cycle; drive the stored-credential worklist to zero with `MIM365-Find-StoredCredentials.ps1`.

Order of operations

`Enable identity (system/user-assigned) → grant least-privilege app-role + consent → Logic App calls Graph with the identity → Function calls Graph with the identity → validate least-privilege + token flow → migrate secret flows and retire credentials → expand`

5. Testing Guide — Managed Identities for Azure Logic Apps & Functions

Run after Implementation Step 5 (validate) and again after each migration (Step 6) and expansion (Step 7). Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names. Each test is **PASS/FAIL**.

Test 1 — Managed identity enabled on the compute

Do: `portal.azure.com` → the Logic App `Graph Automation LogicApp – Contoso – Pilot` and the Function App `Graph Automation FuncApp – Contoso – Pilot` → **Identity** blade; or run `MIM365-Validate.ps1`. **Expected (PASS):** Each resource has a **system-assigned** identity **On**, or the user-assigned `Graph Automation Identity – Contoso – Prod` attached. **FAIL:** No identity present → return to Implementation Step 1.

Test 2 — Least-privilege app-role granted (and nothing broader)

Do: Run `MIM365-Validate.ps1`; or Entra → **Enterprise applications** → the managed identity → **Permissions**. **Expected (PASS):** The identity holds exactly the intended application permission (e.g. `Mail.Read`, recorded as `Mail.Read App Grant – Graph Automation Identity – Prod`) and **no additional** application permissions. **FAIL:** Missing grant, or extra permissions beyond least privilege → fix in Implementation Step 2.

Test 3 — Logic App authenticates to Graph with the identity

Do: Trigger `Graph Automation LogicApp – Contoso – Pilot`; inspect the HTTP action run. **Expected (PASS):** The HTTP action's **Authentication type = Managed identity**, no connector/secret is used, and the call returns **200** with the expected Graph payload. **FAIL:** 401/403, or a stored connection is in use → check the grant (Test 2), the audience, and that Authentication type is Managed identity.

Test 4 — Azure Function authenticates to Graph with the identity

Do: Invoke `Graph Automation FuncApp – Contoso – Pilot`. **Expected (PASS):** The Function returns the expected Graph result using `DefaultAzureCredential` / `ManagedIdentityCredential`, with **no secret in app settings or code**. **FAIL:** Credential does not resolve, or 401/403 → confirm the identity is attached and (for user-assigned) that the correct **Client ID** is passed; check the granted app-role.

Test 5 — No stored credential remains on migrated flows

Do: Run `MIM365-Find-StoredCredentials.ps1` against the migrated workflow. **Expected (PASS):** The workflow no longer appears on the stored-credential worklist — no saved service-account connection and no client secret. **FAIL:** The workflow still lists a stored credential → complete the credential removal in Implementation Step 6.

Test 6 — Secret-based flow migrated with parity

Do: Compare the migrated flow's output against the original secret-based flow for the same input (the Graph result it produced before cutover). **Expected (PASS):** The managed-identity flow returns the **same result** the secret-based flow did — functional parity — and the stored credential has been retired. **FAIL:** Behaviour differs → keep the rollback in place, reconcile the Graph call, and re-test before removing the credential.

Test 7 — Least-privilege boundary holds

Do: From the identity, attempt a Graph call the granted permission does **not** authorize (e.g. writing mail when only `Mail.Read` is granted, or reading a mailbox excluded by the #4 scoping). **Expected (PASS):** The unauthorized call is **denied** (403 / insufficient privileges) — proof the identity is bounded to least privilege (and, for mailboxes, to the #4 scope). **FAIL:** The call succeeds → the identity is over-permissioned; remove the extra grant (Step 2) and apply #4 mailbox scoping.

Test 8 — Estate inventory report

Do: Run `MIM365-Report.ps1`. **Expected (PASS):** A clear identity/permission inventory across the estate (each Logic App/Function, its identity, its app-roles) plus a CSV, with the stored-credential worklist trending toward zero. **FAIL:** Report empty or errors → confirm Microsoft.Graph connectivity and scopes.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–4 pass — the identity authenticates to Graph from **both** a Logic App and a Function, least-privilege confirmed
Migration pass	Tests 5–6 pass — a secret-based flow is migrated with parity and its stored credential retired
Least-privilege pass	Test 7 passes — an unauthorized call is denied

Gate	Requirement
Expand pass	Tests 1–3/4, 8 pass on each newly onboarded workflow before the next expansion

6. Troubleshooting Guide — Managed Identities for Azure Logic Apps & Functions

Use `MIM365-Discovery.ps1` / `MIM365-Validate.ps1` for identity and permission state, `MIM365-Report.ps1` for the estate inventory, and `MIM365-Find-StoredCredentials.ps1` for the migration worklist. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names. Version/feature/endpoint specifics below are flagged _**verify against current Microsoft documentation**_ — do not treat them as asserted fact.

A. Managed identity not enabled / not found

Symptom	Cause	Remediation
HTTP action or code has no identity to use	System-assigned not turned on, or user-assigned not attached	Implementation Step 1: set **Identity → System assigned = On**, or attach `Graph Automation Identity – Contoso – Prod`; confirm with `MIM365-Discovery.ps1`
User-assigned identity attached but token still fails	Code/action did not select the specific identity	For a user-assigned identity, pass its **Client ID** (`ManagedIdentityClientId`) in code, or select it explicitly in the Logic App action

B. Permission / admin consent missing

Symptom	Cause	Remediation
Call returns 403 / "insufficient privileges"	The application app-role was never granted, or consent not given	Run `MIM365-Set-AppRole.ps1 -ManagedIdentityObjectId <id> -GraphAppRole '<perm>' -Execute`; grant admin consent. Requires Privileged Role Administrator / Global Admin
App-role assignment succeeded but access still denied	Consent/propagation delay, or wrong permission name	Allow propagation; re-check with `MIM365-Validate.ps1`. *Verify the exact permission name/scope against current Microsoft documentation — do not assume it.*

C. 403 / insufficient privileges on a mailbox call

• The application permission may be granted but **tenant-wide and not scoped**, or scoped to a **different** mailbox than the call targets. Application mail permissions are constrained by companion product **#4 "Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online."**

• Confirm the mailbox the workflow targets is **within** the #4 scope (Application Access Policy or Graph-for-Exchange RBAC scope). *Verify which scoping model applies to your tenant against current Microsoft documentation.* This guide grants least privilege; #4 owns the mailbox scoping.

D. Token audience / scope errors

Symptom	Cause	Remediation
"Invalid audience" / "wrong resource"	Token requested for the wrong resource/audience	Logic App action: set **Audience =** `https://graph.microsoft.com`. Function: request `https://graph.microsoft.com/.default`. *Verify the audience/resource URI and Graph host for your cloud against current Microsoft documentation*
Works in commercial, fails in a national cloud	Sovereign-cloud endpoints differ (Graph host, authority, audience)	Target the correct sovereign endpoints and set the credential's authority for that cloud. *Verify the exact `.us` (or other) endpoints against current Microsoft documentation*

E. User- vs system-assigned confusion

• **Symptom:** a Function using `DefaultAzureCredential` picks the wrong identity, or fails when two identities are present.

• **Cause:** with a user-assigned identity attached, `DefaultAzureCredential` needs the **Client ID** to disambiguate; without it, it may not resolve the intended identity.

• **Remediation:** pass `ManagedIdentityClientId = "<user-assigned Client ID>"` for a user-assigned identity, or omit it and rely on system-assigned. Do not enable both and leave the code ambiguous. Confirm which identity holds the grant with `MIM365-Validate.ps1`.

F. Azure Functions credential not resolving

Symptom	Cause	Remediation
`DefaultAzureCredential` throws "no credential available"	Running locally (no managed identity), or identity not enabled on the Function App	In Azure, ensure the identity is enabled/attached (Step 1). Locally, `DefaultAzureCredential` falls back to developer credentials — expected; test in Azure

Symptom	Cause	Remediation
Credential resolves but Graph SDK call fails to compile / run	SDK package/class/method names differ by version	*Verify the current Azure.Identity and Microsoft.Graph SDK package names, classes, and method signatures against current Microsoft documentation* — they change between versions

G. Connector action won't use a managed identity

- Not every **Office 365 Outlook** connector action supports managed-identity auth. *Verify which connector operations support managed identity against current Microsoft documentation.*
- **Remediation:** use the **direct Graph HTTP call** (Implementation Step 3) instead of the connector — the robust, secret-free pattern. The mailbox siblings #1 (replace-the-connector), #2 (change-notifications), and #3 (polling) show these Graph calls end-to-end.

H. Stored credential still flagged after migration

- `MIM365-Find-StoredCredentials.ps1` still lists a migrated workflow → the old **API connection** or **secret app setting** was not removed. Complete Implementation Step 6: delete the saved connection / secret app setting once managed-identity parity is confirmed, then re-run the finder — it should drop off.

I. Diagnostics quick reference

```
# Which compute has an identity + what app-roles it holds:
.\MIM365-Discovery.ps1

# PASS/FAIL a specific resource against expected least privilege:
.\MIM365-Validate.ps1 -ResourceName 'Graph Automation LogicApp - Contoso - Pilot' -ResourceGroup '<rg>' -E

# Estate inventory + CSV:
.\MIM365-Report.ps1

# Migration worklist (who still uses a stored credential):
.\MIM365-Find-StoredCredentials.ps1

# Grant one least-privilege app-role (preview first, then -Execute):
.\MIM365-Set-AppRole.ps1 -ManagedIdentityObjectId '<identity object id>' -GraphAppRole 'Mail.Read'
```

J. When to escalate

If the identity is enabled, the expected app-role is granted and consented, the audience/endpoints are correct for the cloud, and a Graph call still fails after the normal propagation window, collect `MIM365-Report.ps1` output and the identity/permission details and open a Microsoft support case. *Confirm current permission names, connector support, Consumption-vs-Standard behaviour, and sovereign-cloud endpoints against current Microsoft documentation before escalating.*

7. Best Practices — Managed Identities for Azure Logic Apps & Functions

• **Remove the credential, don't rotate it.** The whole point is that a managed identity has nothing to store, rotate, or leak. Treat any stored service-account connection or client secret in a workflow as debt to be migrated, not as a thing to manage better.

• **Least privilege by default.** Grant the managed identity the **one** application permission the task needs and nothing else. A workflow that only reads mail gets `Mail.Read`, not `Mail.ReadWrite` and not `Directory.Read.All` "to be safe." *Verify the exact permission for your call against current Microsoft documentation.*

• **Scope mailbox permissions — always.** An application mail permission is tenant-wide until you constrain it. Never leave `Mail.Read` (or similar) unscoped in production — restrict it to the specific mailbox(es) per companion product #4. Granting the permission is not the same as scoping it.

• **Prefer a shared user-assigned identity for an estate.** One identity attached to many workflows means one set of grants to audit, one object to review, and one thing to remove — far simpler than a system-assigned identity (and its own grants) on every resource. Use system-assigned for genuinely one-off, self-contained workflows.

• **Call Graph directly rather than lean on legacy connectors.** Not every Office 365 / Outlook connector action supports managed-identity auth; the direct Graph HTTP call (Logic Apps) or SDK call (Functions) is the robust, secret-free pattern. *Verify connector managed-identity support against current Microsoft documentation.*

• **Name everything and reuse the name.** Use `<Purpose> – <Scope> – <Stage>` for the identity, the Logic App, the Function, and label the grant, and reuse those names verbatim across scripts, tests, and rollback so the estate stays auditable.

• **Pilot one workflow, then expand.** Prove the pattern on a single Logic App/Function before rolling it across the estate — the same enable-grant-call-validate flow every time, so each migration is familiar and low-risk.

• **Keep a rollback for every cutover.** Before removing a stored credential, confirm the migrated flow has parity and keep the previous connection available until it does. Retire the credential only after the managed-identity flow is proven.

• **Match the cloud's endpoints.** For national/sovereign clouds, the Graph host, authority, and token audience differ from commercial — set them correctly in the Logic App action and the Function's credential. *Verify the endpoints for your cloud against current Microsoft documentation.*

• **Inventory continuously.** Run the stored-credential finder and the estate report each cycle; a shrinking worklist and a clean least-privilege inventory are the evidence the pattern is taking hold.

8. Deliverables Summary — Managed Identities for Azure Logic Apps & Functions

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What managed identities are, system- vs user-assigned, the Logic Apps + Functions patterns, pointers to #4 (mailbox scoping) and #1/#2/#3 (mailbox uses); benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Azure Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — enable identity, grant least-privilege app-role + consent, Logic App Graph call, Function Graph call (code example), validate, migrate
5. Testing Guide	PASS/FAIL validation for both compute types, least privilege, and migration parity
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts (Microsoft.Graph)

Script	What it does
`MIM365-Discovery.ps1`	Read-only — enumerates Logic Apps/Functions and whether each has a managed identity and which application app-roles are assigned
`MIM365-Validate.ps1`	PASS/FAIL — target compute has a managed identity with least-privilege app permissions and no leftover secret-based connection
`MIM365-Report.ps1`	Identity/permission inventory across the estate, plus a CSV export
`MIM365-Find-StoredCredentials.ps1`	Read-only — reports workflows/apps still using stored service-account connections or client secrets (the migration worklist)
`MIM365-Set-AppRole.ps1`	The only state-changer — assigns/verifies an application app-role to a managed identity; **defaults to -WhatIf**

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the standard model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — piloting one workflow before expanding, with a stored-credential worklist as the migration

driver and a rollback for every cutover.

Outcome

Azure automations authenticating to Microsoft 365 with **no stored credential** — Logic Apps and Functions reaching Microsoft Graph through managed identities and least-privilege application permissions, with existing secret-based flows migrated off and retired.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MIM365-Discovery.ps1

Purpose: Read-only discovery for the "Managed Identities for Azure Logic Apps & Functions" solution. Enumerates the Logic Apps and Function Apps in the subscription, reports each resource's managed identity (system-assigned principal and/or attached user-assigned identities), and for each identity lists the application app-role assignments (e.g. Mail.Read) granted on Microsoft Graph. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MIM365-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\MIM365-Discovery.ps1 -SubscriptionId '<sub id>' Requires Az PowerShell (Az.Accounts, Az.Resources) to enumerate compute + identities, and the Microsoft.Graph SDK to resolve app-role assignments. Run Connect-AzAccount and Connect-MgGraph first, or the script will prompt. GRAPH SCOPES (least privilege, read-only) Application.Read.All, AppRoleAssignment.Read.All, Directory.Read.All (Connect-MgGraph -Scopes 'Application.Read.All','AppRoleAssignment.Read.All','Directory.Read.All') EXPECTED RESULTS A per-resource snapshot: each Logic App / Function App, its managed-identity kind (system / user-assigned / none), and the Graph application app-roles assigned to each identity — the readiness picture before you grant permissions or migrate secret-based flows. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers only; makes no changes to identities, permissions, or workflows. White-label sample tenant: contoso.com. > [CmdletBinding()] param([string]$SubscriptionId)

How to run: powershell -ExecutionPolicy Bypass -File .\MIM365-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\MIM365-Discovery.ps1 -SubscriptionId '<sub id>' Requires Az PowerShell (Az.Accounts, Az.Resources) to enumerate compute + identities, and the Microsoft.Graph SDK to resolve app-role assignments. Run Connect-AzAccount and Connect-MgGraph first, or the script will prompt. GRAPH SCOPES (least privilege, read-only) Application.Read.All, AppRoleAssignment.Read.All, Directory.Read.All (Connect-MgGraph -Scopes 'Application.Read.All','AppRoleAssignment.Read.All','Directory.Read.All') EXPECTED RESULTS A per-resource snapshot: each Logic App / Function App, its managed-identity kind (system / user-assigned / none), and the Graph application app-roles assigned to each identity — the readiness picture before you grant permissions or migrate secret-based flows. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers only; makes no changes to identities, permissions, or workflows. White-label sample tenant: contoso.com. > [CmdletBinding()] param([string]$SubscriptionId)

MIM365-Find-StoredCredentials.ps1

Purpose: Read-only migration triage for the "Managed Identities for Azure Logic Apps & Functions" solution. Flags the workflows that still depend on a stored credential rather than a managed identity, by reporting: - Logic Apps referencing an API connection (Microsoft.Web/connections) — e.g. a saved Office 365 Outlook / service-account connection; - Function Apps whose Application settings contain likely secret keys (names matching SECRET*, *PASSWORD*, *PWD*, *CLIENTSECRET*, *CONNECTIONSTRING* — the value is NOT read or printed, only the presence of the key); - resources with NO managed identity enabled (cannot yet use the credential-free pattern). Read-only: it inventories, it does not change or remove anything. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MIM365-Find-StoredCredentials.ps1 powershell -ExecutionPolicy Bypass -File .\MIM365-Find-StoredCredentials.ps1 -SubscriptionId '<sub id>' -OutputCsv

'.\MIM365-migration-worklist.csv' Requires Az.Accounts, Az.Resources, Az.Websites. Run Connect-AzAccount first, or it will prompt. GRAPH SCOPES None — this script queries Azure Resource Manager only (no Microsoft Graph call). It never reads or outputs secret VALUES, only the presence of stored-credential indicators. EXPECTED RESULTS A console table and CSV of the migration worklist: each resource still using a stored credential or lacking a managed identity, with the reason. Re-run after each migration; a migrated workflow drops off the list. .NOTES Azure Solutions — read-only. The repeatable way. Reports indicators only; never reads, logs, or prints secret values. White-label sample tenant: contoso.com. > [CmdletBinding()] param([string]$SubscriptionId, [string]$OutputCsv = ".\MIM365-migration-worklist.csv")

How to run: powershell -ExecutionPolicy Bypass -File .\MIM365-Find-StoredCredentials.ps1 powershell -ExecutionPolicy Bypass -File .\MIM365-Find-StoredCredentials.ps1 -SubscriptionId '<sub id>' -OutputCsv '.\MIM365-migration-worklist.csv' Requires Az.Accounts, Az.Resources, Az.Websites. Run Connect-AzAccount first, or it will prompt. GRAPH SCOPES None — this script queries Azure Resource Manager only (no Microsoft Graph call). It never reads or outputs secret VALUES, only the presence of stored-credential indicators. EXPECTED RESULTS A console table and CSV of the migration worklist: each resource still using a stored credential or lacking a managed identity, with the reason. Re-run after each migration; a migrated workflow drops off the list. .NOTES Azure Solutions — read-only. The repeatable way. Reports indicators only; never reads, logs, or prints secret values. White-label sample tenant: contoso.com. > [CmdletBinding()] param([string]$SubscriptionId, [string]$OutputCsv = ".\MIM365-migration-worklist.csv")

MIM365-Report.ps1

Purpose: Reporting for the "Managed Identities for Azure Logic Apps & Functions" solution. Walks every Logic App and Function App in the subscription and produces one row per (resource, identity, app-role): the resource name and type, the managed-identity kind (system / user-assigned / none), the identity's principalId, and each application Graph app-role it holds. Writes a CSV for auditing and trend tracking. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MIM365-Report.ps1 powershell -ExecutionPolicy Bypass -File .\MIM365-Report.ps1 -SubscriptionId '<sub id>' -OutputCsv '.\MIM365-identity-inventory.csv' Requires Az.Accounts, Az.Resources and the Microsoft.Graph SDK. Run Connect-AzAccount and Connect-MgGraph first, or the script will prompt. GRAPH SCOPES (least privilege, read-only) Application.Read.All, AppRoleAssignment.Read.All, Directory.Read.All EXPECTED RESULTS A console table and a CSV (default .\MIM365-identity-inventory.csv) listing each Logic App / Function App, its managed-identity kind, principalId, and granted Graph application app-roles — the estate inventory to audit least privilege and track drift each cycle. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers only; makes no changes. White-label sample tenant: contoso.com. > [CmdletBinding()] param([string]$SubscriptionId, [string]$OutputCsv = ".\MIM365-identity-inventory.csv")

How to run: powershell -ExecutionPolicy Bypass -File .\MIM365-Report.ps1 powershell -ExecutionPolicy Bypass -File .\MIM365-Report.ps1 -SubscriptionId '<sub id>' -OutputCsv '.\MIM365-identity-inventory.csv' Requires Az.Accounts, Az.Resources and the Microsoft.Graph SDK. Run Connect-AzAccount and Connect-MgGraph first, or the script will prompt. GRAPH SCOPES (least privilege, read-only) Application.Read.All, AppRoleAssignment.Read.All, Directory.Read.All EXPECTED RESULTS A console table and a CSV (default .\MIM365-identity-inventory.csv) listing each Logic App / Function App, its managed-identity kind, principalId, and granted Graph application app-roles — the estate inventory to audit least privilege and track drift each cycle. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers only; makes no changes. White-label sample tenant: contoso.com. >

[CmdletBinding()] param([string]$SubscriptionId, [string]$OutputCsv = ".\MIM365-identity-inventory.csv")

MIM365-Set-AppRole.ps1

Purpose: Grants a single application Graph permission to a managed identity's service principal by creating an app-role assignment — the equivalent of granting the application permission and admin-consenting it for that identity. Grant ONE least-privilege role at a time (e.g. Mail.Read); never a broad tenant-wide grant. If the role is already assigned, it reports that and makes no change (idempotent / verify mode). DEFAULTS TO -WhatIf — it prints the grant it WOULD make and changes NOTHING until you pass -Execute. This is stated here and echoed at run time. For MAILBOX permissions, granting is only half the job: scope the grant to specific mailboxes per companion product #4 ("Scoping Application-Permission Mailbox Access") before production. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\MIM365-Set-AppRole.ps1 -ManagedIdentityObjectId '<identity object id>' -GraphAppRole 'Mail.Read' Apply for real (explicit): powershell -ExecutionPolicy Bypass -File .\MIM365-Set-AppRole.ps1 -ManagedIdentityObjectId '<identity object id>' -GraphAppRole 'Mail.Read' -Execute Requires the Microsoft.Graph SDK. Run Connect-MgGraph first, or the script will prompt. GRAPH SCOPES (required to READ app-roles and WRITE the assignment) Application.Read.All, AppRoleAssignment.ReadWrite.All, Directory.Read.All (Granting an application permission / app-role assignment is a privileged, admin-consented action — verify the exact role name/scope for your call against current Microsoft documentation.) EXPECTED RESULTS In -WhatIf mode (default): a printed description of the app-role that WOULD be granted — no change. With -Execute: the app-role assignment is created (or reported as already present). .NOTES Azure Solutions — DEFAULTS TO -WhatIf; the only state-changing script here. The repeatable way. Grants exactly one least-privilege application app-role to the named identity; touches nothing else. Mailbox scoping is out of scope here — see companion product #4. White-label sample tenant: contoso.com. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ManagedIdentityObjectId, # the managed identity's service principal Object ID [string]$GraphAppRole = 'Mail.Read', # the ONE least-privilege application permission to grant [switch]$Execute # omit for -WhatIf (default safe mode))

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\MIM365-Set-AppRole.ps1 -ManagedIdentityObjectId '<identity object id>' -GraphAppRole 'Mail.Read' Apply for real (explicit): powershell -ExecutionPolicy Bypass -File .\MIM365-Set-AppRole.ps1 -ManagedIdentityObjectId '<identity object id>' -GraphAppRole 'Mail.Read' -Execute Requires the Microsoft.Graph SDK. Run Connect-MgGraph first, or the script will prompt. GRAPH SCOPES (required to READ app-roles and WRITE the assignment) Application.Read.All, AppRoleAssignment.ReadWrite.All, Directory.Read.All (Granting an application permission / app-role assignment is a privileged, admin-consented action — verify the exact role name/scope for your call against current Microsoft documentation.) EXPECTED RESULTS In -WhatIf mode (default): a printed description of the app-role that WOULD be granted — no change. With -Execute: the app-role assignment is created (or reported as already present). .NOTES Azure Solutions — DEFAULTS TO -WhatIf; the only state-changing script here. The repeatable way. Grants exactly one least-privilege application app-role to the named identity; touches nothing else. Mailbox scoping is out of scope here — see companion product #4. White-label sample tenant: contoso.com. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ManagedIdentityObjectId, # the managed identity's service principal Object ID [string]$GraphAppRole = 'Mail.Read', # the ONE least-privilege application permission to grant [switch]$Execute # omit for -WhatIf (default safe mode))

MIM365-Validate.ps1

Purpose: Validation for the "Managed Identities for Azure Logic Apps & Functions" solution. For a named Logic App or Function App, confirms: (1) it has a managed identity; (2) that identity holds the expected application app-role (e.g. Mail.Read) on Microsoft Graph; (3) it holds NOTHING broader than the expected set (least privilege); and (4) — for a Logic App — no stored/secret-based API connection remains in its resource group. Prints PASS/FAIL per check. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MIM365-Validate.ps1 -ResourceName 'Graph Automation LogicApp - Contoso - Pilot' -ResourceGroup '<rg>' -ExpectedAppRoles 'Mail.Read' powershell -ExecutionPolicy Bypass -File .\MIM365-Validate.ps1 -ResourceName 'Graph Automation FuncApp - Contoso - Pilot' -ResourceGroup '<rg>' -ExpectedAppRoles 'Mail.Read' -SubscriptionId '<sub id>' Requires Az.Accounts, Az.Resources and the Microsoft.Graph SDK. Run Connect-AzAccount and Connect-MgGraph first, or the script will prompt. GRAPH SCOPES (least privilege, read-only) Application.Read.All, AppRoleAssignment.Read.All, Directory.Read.All EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. Least privilege FAILS if the identity holds any application app-role not in -ExpectedAppRoles; the secret check FAILS if a stored API connection is still present for a Logic App. .NOTES Azure Solutions — read-only PASS/FAIL. The repeatable way. Reports on configuration only; makes no changes. Mailbox-permission scoping is validated separately per companion product #4. White-label sample tenant: contoso.com. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceName, [Parameter(Mandatory)] [string]$ResourceGroup, [string[]]$ExpectedAppRoles = @('Mail.Read'), [string]$SubscriptionId)

How to run: powershell -ExecutionPolicy Bypass -File .\MIM365-Validate.ps1 -ResourceName 'Graph Automation LogicApp - Contoso - Pilot' -ResourceGroup '<rg>' -ExpectedAppRoles 'Mail.Read' powershell -ExecutionPolicy Bypass -File .\MIM365-Validate.ps1 -ResourceName 'Graph Automation FuncApp - Contoso - Pilot' -ResourceGroup '<rg>' -ExpectedAppRoles 'Mail.Read' -SubscriptionId '<sub id>' Requires Az.Accounts, Az.Resources and the Microsoft.Graph SDK. Run Connect-AzAccount and Connect-MgGraph first, or the script will prompt. GRAPH SCOPES (least privilege, read-only) Application.Read.All, AppRoleAssignment.Read.All, Directory.Read.All EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. Least privilege FAILS if the identity holds any application app-role not in -ExpectedAppRoles; the secret check FAILS if a stored API connection is still present for a Logic App. .NOTES Azure Solutions — read-only PASS/FAIL. The repeatable way. Reports on configuration only; makes no changes. Mailbox-permission scoping is validated separately per companion product #4. White-label sample tenant: contoso.com. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceName, [Parameter(Mandatory)] [string]$ResourceGroup, [string[]]$ExpectedAppRoles = @('Mail.Read'), [string]$SubscriptionId)